

2.2 CloudWatch: how you find out what went wrong

i In this section we are learning where to look when something breaks in production in AWS. “When something breaks in production, this is where you look. Learning it now, calmly, on your own toy code is much easier than learning it during a live incident with a pharmacy waiting.”

This task is mainly about learning **how to investigate a Lambda when something goes wrong.**

Learning objectives

What are a log group and a log stream, and how do they map to your function?

AWS CloudWatch Logs provides a scalable, reliable way to collect and monitor log data from your applications and infrastructure. There are two parts of CloudWatch Logs: **log streams** and **log groups**.

Log stream

DEF: A **log stream** is an ordered sequence of log events that all originate from the same source—for example, an EC2 instance, Lambda function, or on-premises server. Each event carries a timestamp and a message, forming a timeline of activity for that component. Common log stream contents include:

- Application error messages
- User access and transaction logs
- Performance and diagnostics data

Log groups

DEF: A **log group** acts as a container for one or more log streams. Think of it as a logical folder that centralizes related streams under common settings and policies. Within a log group, you can configure:

- **Retention period:** How long to keep log data
- **Metric filters:** Generate CloudWatch metrics from log patterns
- **Access control:** IAM policies governing who can view or manage logs

Concept	Definition	Key Use Cases
Log Stream	Timestamped sequence of events from a single source	Troubleshooting individual instances or components
Log Group	Container for multiple streams with shared settings	Centralized management, retention, and monitoring

What happens to `console.log()` in a Lambda?

When you use `console.log()` in an AWS Lambda function, the output is automatically captured and sent to **AWS CloudWatch Logs**.

How do you read a Lambda error and its stack trace?

You read it by going to the CloudWatch log — you can also see it in the terminal, but you get more information in the CloudWatch log. You can see the stack trace in the CloudWatch log as well. The **stack trace** tells you things such as:

- What error happened
- Where did it happen?
- Which file?
- Which line?

The stack trace is usually shown underneath the error log in CloudWatch (you click the drop-down on the log to see it).

Which metrics does Lambda give you for free — invocations, errors, duration, throttles — and what would each one tell you? (understanding lambda metrics)

Lambda automatically publishes several useful metrics to CloudWatch. Four metrics your ticket specifically wants you to understand are **Invocations, Errors, Duration and Throttles**.

Metric	Simple meaning	What it could tell you
Invocations	Number of times the function ran	"Is my Lambda actually being called?"
Errors	Number of invocations that failed	"Has something started breaking?"
Duration	How long the function takes to run	"Has my Lambda become unusually slow?"

Throttles	Requests Lambda couldn't process because execution capacity wasn't available	"Are too many requests arriving or are we hitting concurrency limits?"
------------------	--	--

For example, imagine:

```

1 Yesterday:
2 Invocations: 1,000
3 Errors: 2
4
5 Today:
6 Invocations: 1,000
7 Errors: 700

```

Something clearly went wrong.

Or:

```

1 Normal Duration:
2 100ms
3
4 Current Duration:
5 8,000ms

```

Your function might not technically be failing, but something is making it much slower.

AWS defines **Errors** as invocations resulting in function errors, including code exceptions and runtime errors; **Throttles** counts rejected invocation requests when Lambda cannot provide available concurrency.

What is Logs Insights for?

Logs Insights is used for finding specific logs when we are working at a production level, where there could be between 10 and 1,000,000 logs. Manually looking through them to find one specific log is nearly impossible at this scale. Therefore we use Logs Insights, which runs queries to find aspects of a log, e.g. date, name, error names, etc., to find the exact log you are looking for in little to no time.

Task



- *Do*
 - Add useful logging to your Lambda. Think about what you'd want to see at 6 a.m. when something's broken.
 - Make it throw an error on purpose.

- Find that error in CloudWatch.
- Use Logs Insights to search your logs for it.
- *Done when*
 - You've found your own deliberate error in CloudWatch and screenshot it into the worksheet.
 - You can say where you'd look *first* if a Lambda started failing in production.

Okay — now we are making code that will give a better error message in the CloudWatch Logs. This is important because:

Imagine someone tells you at 6 a.m.:

"FRIDGE-001 isn't working."

Seeing:

```
1 fridgeId: FRIDGE-001
2 minTempC: 3
3 maxTempC: 7
4 status: ok
```

Is much more useful than seeing a standard log saying:

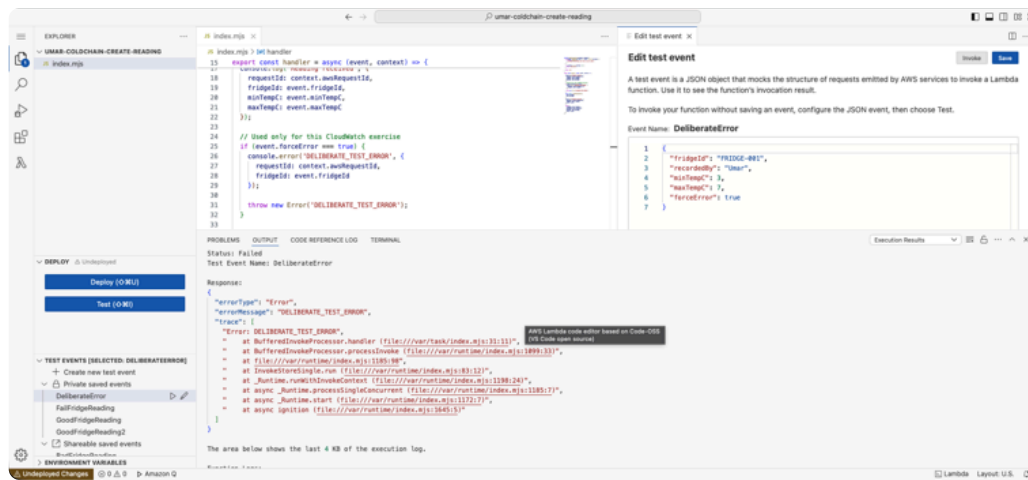
```
1 Lambda ran
```

This is just for this section: we will be using this IF statement (potential). I am not too sure though.

So first we add this IF block to the already existing code.

```
1 // Used only for this CloudWatch exercise
2 if (event.forceError === true) {
3   console.error('DELIBERATE_TEST_ERROR', {
4     requestId: context.awsRequestId,
5     fridgeId: event.fridgeId
6   });
7
8   throw new Error('DELIBERATE_TEST_ERROR');
9 }
```

This IF block — when we ran the test `DeliberateError` — the value “forceError” = true was added which, like the code title says, would deliberately cause an error. This is shown in the screenshot below:



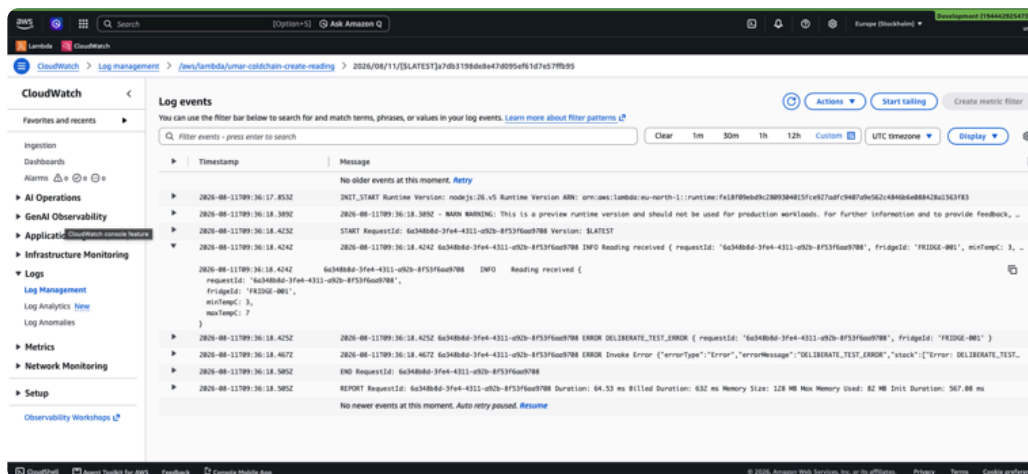
The test was a success even though the result was an error because our expected result was an error.

```

1  const status = getReadingStatus(
2      event.minTempC,
3      event.maxTempC
4  );
5
6  console.log('Reading processed', {
7      requestId: context.awsRequestId,
8      fridgeId: event.fridgeId,
9      status: status
10 });

```

The code above is the second new addition to the function; it will give a more descriptive CloudWatch log. This goes below the IF block. So when we navigate to CloudWatch and look at the /aws/lambda/umar-coldchain-create-reading stream logs,



You can see in the screenshot above the error is showing which fridge failed and what temps caused it to fail.

So now the full code will look like:

```

1  function getReadingStatus(minTempC, maxTempC) {
2      if (!Number.isFinite(minTempC) || !Number.isFinite(maxTempC)) {

```

```

3     throw new TypeError(
4       'minTempC and maxTempC must both be valid numbers'
5     );
6   }
7
8   if (minTempC < 2 || maxTempC > 8) {
9     return 'excursion';
10  }
11
12  return 'ok';
13 }
14
15 export const handler = async (event, context) => {
16
17   console.log('Reading received', {
18     requestId: context.awsRequestId,
19     fridgeId: event.fridgeId,
20     minTempC: event.minTempC,
21     maxTempC: event.maxTempC
22   });
23
24   // Used only for this CloudWatch exercise
25   if (event.forceError === true) {
26     console.error('DELIBERATE_TEST_ERROR', {
27       requestId: context.awsRequestId,
28       fridgeId: event.fridgeId
29     });
30
31     throw new Error('DELIBERATE_TEST_ERROR');
32   }
33
34   const status = getReadingStatus(
35     event.minTempC,
36     event.maxTempC
37   );
38
39   console.log('Reading processed', {
40     requestId: context.awsRequestId,
41     fridgeId: event.fridgeId,
42     status: status
43   });
44
45   return {
46     fridgeId: event.fridgeId,
47     recordedBy: event.recordedBy,
48     minTempC: event.minTempC,
49     maxTempC: event.maxTempC,
50     status: status
51   };
52 };

```

NOTE: The part underneath the error is the **stack trace**.

It tells you things such as:

- What error happened
- Where did it happen?
- Which file?
- Which line?

LUKE EXPLANATION: CloudWatch logs configuration-related errors.

When making something in VS Code — for example, a login button — VS Code would not pick up configuration errors such as wrong API keys, because the code itself is valid and would run without errors when the button is clicked.

However, you still wouldn't be able to access the account because the API key is wrong. That is when you would go to CloudWatch for configuration errors. CloudWatch will show the stack trace and you will see that the API key is wrong.

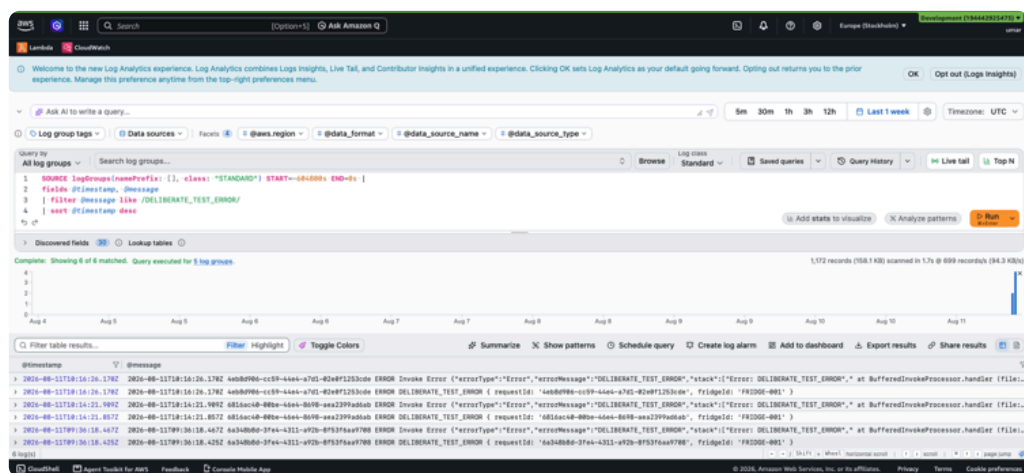
OVERALL: CloudWatch Logs surfaces configuration errors; whereas VS Code will show coding errors.

Using Logs Insights

Finding 1 log manually when you have 5 logs is easy. But at production scale there could be 10 to 1,000,000 logs and it becomes almost impossible to manually find a single log. This is why Logs Insights exists.

That's why **CloudWatch Logs Insights** exists: it lets you query and analyse log data rather than manually opening individual streams. AWS provides Logs Insights query language specifically for searching and analysing CloudWatch log data.

Below is an image of what the console will look like;



You run commands to locate certain logs with certain values to make sure you get the correct ones. The image above shows this with the query and the orange Run button. Then at the

bottom are the logs that matched the query you ran.

```
1 fields @timestamp, @message
2 filter @message like /DELIBERATE_TEST_ERROR/
3 sort @timestamp desc
```

Above is the command I ran.

Navigating to CloudWatch

Stream logs

CloudWatch → Log management → Click on the function you are working on → Open the log stream you are looking for (probably the latest stream at the top).

Logs Insights

CloudWatch → Logs → Log analytics

Task question

"You can say where you'd look first if a Lambda started failing in production."

If a Lambda started failing in production, I would first check CloudWatch. I would use the Lambda metrics to confirm whether errors had increased and how many invocations were affected, then open the function's CloudWatch log group and inspect the latest logs. I would look for the error message and stack trace because these can show what failed and where in the Lambda code the failure occurred. If there were large amounts of logs, I would use CloudWatch Logs Insights to search and filter them for the specific error, request or function invocation.

 Questions / Queries / Pushback

No worries for this section so far.